ENGINEERING BRIEF / V1.0

# Bangun Sistem Zero Trust yang Benar-Benar Jalan

Instruksi implementasi penuh untuk AI coding agent: UI premium, API Gateway, PostgreSQL lokal, JWT lifecycle, service-to-service security, serta pengujian menyeluruh.

MANDAT UTAMA: jangan berhenti pada rencana atau mockup. Audit workspace, rancang, implementasikan, jalankan, test, perbaiki, dan laporkan bukti hasilnya.

Produk kerja: Sistem Nilai Akademik berbasis Zero Trust
Database: PostgreSQL lokal  Frontend: React + TypeScript  Backend: Node.js + TypeScript
Prioritas: keamanan dan pemenuhan rubrik dahulu; polish visual dan fitur plus sesudah baseline lulus.

# 01 / Mandat dan cara kerja

Anda bertindak sebagai Principal Software Architect, Senior Product Designer, Backend/Security Engineer, PostgreSQL Engineer, dan QA Automation Engineer. Kerjakan di workspace yang saat ini terbuka pada agent. Hasil akhir adalah repo yang bisa dijalankan dan diuji, bukan hanya proposal, scaffold, atau kumpulan komponen tampilan.

## Aturan kerja non-negotiable

- Audit dahulu. Periksa direktori kerja, git status, file yang sudah ada, package manager, versi runtime, environment, dan batasan proyek. Jangan menimpa pekerjaan pengguna atau menjalankan perintah destruktif tanpa alasan.
- Gunakan sumber. Baca PDF tugas praktikum Zero Trust yang tersedia. Pertahankan komponen, endpoint, role, masa berlaku token, dan aturan akses yang ditentukan dosen. Jika ada konflik antara keinginan tambahan dengan rubrik, rubrik menang.
- Skill wajib dipakai. Pilih skill yang memang ada di instalasi user, baca instruksinya, terapkan, dan catat skill + kontribusi nyatanya. Jangan sekadar menyebut nama skill.
- Jangan bertanya bila tidak perlu. Ambil keputusan teknis yang masuk akal, dokumentasikan asumsi, dan lanjutkan. Berhenti hanya jika ada blocker nyata seperti kredensial/akses yang tidak tersedia atau perubahan yang berisiko merusak data pengguna.
- Implementasi bertahap. Buat kontrak API dan skema data, implementasikan fitur inti, jalankan test, lalu perbaiki kegagalan. Jangan membangun fitur tambahan sebelum persyaratan wajib lulus.
- Bukti, bukan klaim. Jangan mengatakan test lulus, service sehat, atau UI rapi kecuali benar-benar menjalankan test/cek/screenshot dan memeriksa hasilnya.

## Definisi sukses

> Seseorang dapat login, mendapat access JWT yang dibuat server, membaca/mengubah data sesuai role dan konteks, ditolak saat melanggar policy, melihat keputusan tercatat di audit trail, dan menyelesaikan alur finalisasi nilai → sinkronisasi transkrip lewat Gateway. Seluruhnya didukung UI yang matang dan automated tests.

## Hasil akhir yang harus diserahkan agent

- Repository lengkap, file konfigurasi contoh, migration/seed, README yang dapat diikuti dari mesin lokal yang bersih.
- Unit, integration, security, dan Playwright E2E tests; hasil aktual dan daftar isu yang masih terbuka.
- Dokumentasi arsitektur, diagram alur keamanan, API contract, decision log skill, serta rencana demo video 6–9 menit.

# 02 / Rubrik tugas: jangan ada yang terlewat

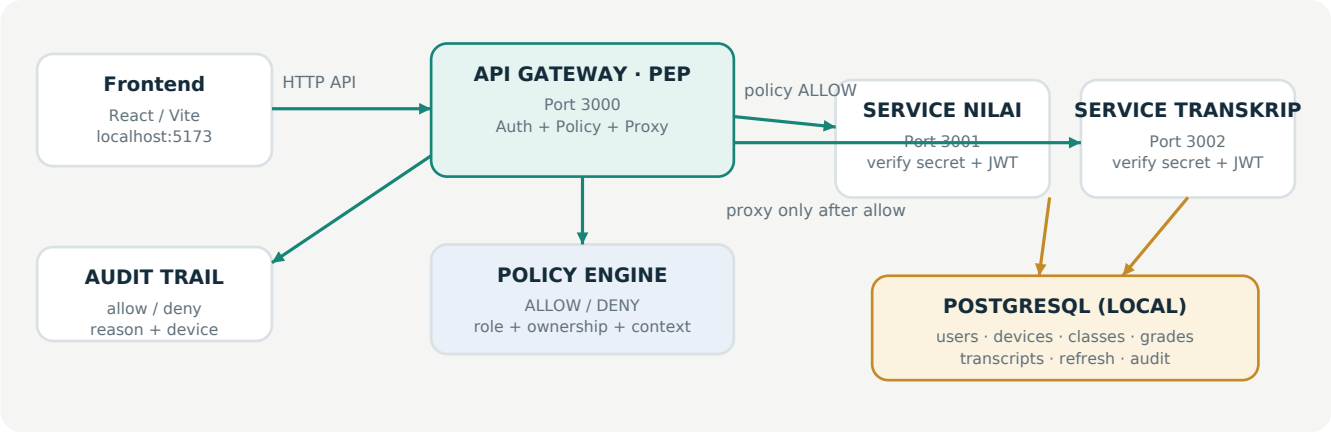
PDF tugas meminta implementasi prinsip never trust, always verify, least privilege, dan assume breach pada level aplikasi, serta API Gateway sebagai PEP yang terpisah dari service. Skenario utama adalah Sistem Nilai: mahasiswa, dosen, admin, Service Nilai, Service Transkrip, satu database bersama, dan audit log.

| Kebutuhan wajib | Ketentuan yang harus nyata di kode | Bukti penerimaan |
|---|---|---|
| API Gateway / PEP | Satu-satunya pintu masuk client pada port 3000; policy decision, audit logging, lalu proxy. | Frontend hanya memanggil Gateway; request tidak lolos policy tidak diteruskan. |
| Service terpisah | Service Nilai port 3001; Service Transkrip port 3002. Keduanya app/proses terpisah. | Health checks dan smoke tests tiap proses; verifikasi secret + token di service. |
| Access JWT | Dibuat saat login berhasil; TTL 10 menit; minimal claim sub dan role. | Token valid diterima; expired/wrong signature/wrong audience ditolak. |
| Refresh token | TTL 1 hari, disimpan di PostgreSQL agar bisa dicabut; logout mencabut token. | Refresh rotation; token revoked/expired/reuse ditolak. |
| Service token | Service Nilai meminta POST /auth/service memakai client ID + secret; TTL 5 menit; role service. | Token user tidak bisa memanggil endpoint sync. |
| Gateway secret | Gateway memasang X-Gateway-Secret ketika meneruskan request. | Direct call tanpa secret ditolak; secret dari client tidak dipercaya. |
| Policy + konteks | Role, kepemilikan NIM/kelas, device terdaftar, nilai belum final, jam 07.00–21.00. | Negative tests untuk setiap aturan; alasan deny tercatat. |
| Audit log | Waktu, subject user/service, endpoint, device, keputusan, alasan. | DB berisi log allow dan deny; secret/token tidak pernah ditulis ke log. |
| Seed/demo | Minimal 1 admin, 2 dosen, 3 mahasiswa; 2 kelas berbeda; satu device terdaftar per user. | README akun demo + device ID dan langkah Postman/UI. |

> Koreksi penting: token JWT tidak boleh “digenerate” ulang pada setiap server restart. Buat signing key sekali saat setup, simpan secara lokal dengan aman, lalu backend menerbitkan token baru setiap login/refresh sesuai TTL. Jangan commit private key atau secret.

# 03 / Arsitektur target

Gunakan arsitektur sederhana tetapi benar-benar terpisah proses. Jangan mengakali requirement dengan menaruh tiga router di satu Express app. Policy Engine/Policy Administrator merupakan modul dalam Gateway; service tetap memiliki pertahanan sendiri. PostgreSQL berjalan lokal.




## Alur request pengguna

1. Frontend mengirim request hanya ke API Gateway: Authorization: Bearer access JWT dan X-Device-ID.
2. Gateway memvalidasi bentuk token, tanda tangan, issuer, audience, expiry, token_use, lalu memeriksa device dan meminta policy decision.
3. Policy Engine membaca atribut database yang perlu untuk memutuskan hak akses (ownership kelas/NIM, status final, waktu). Default adalah DENY.
4. Gateway menulis audit event (allow/deny + reason) tanpa memuat password, JWT, refresh token, atau secret.
5. Jika ALLOW, Gateway membersihkan header sensitif dari request masuk, memasang X-Gateway-Secret dari environment, lalu proxy ke service yang sesuai.

## Alur finalisasi dan sinkronisasi

1. Dosen memfinalisasi kelas yang benar-benar diampu dan masih dalam jam akses. Service Nilai validasi kembali token + gateway secret dan state kelas.
2. Perubahan finalisasi dan event sinkronisasi disimpan secara konsisten. Buat operasi idempotent agar retry tidak menggandakan transkrip.
3. Service Nilai memperoleh service token dari Gateway melalui POST /auth/service (client ID/secret dari env), lalu memanggil endpoint sync melalui Gateway, bukan URL internal Service Transkrip secara langsung.
4. Gateway hanya mengizinkan role/token service untuk endpoint sync. Service Transkrip memverifikasi secret + token service, lalu upsert transkrip.

> Jangan percaya X-Gateway-Secret yang dikirim client. Gateway harus menghapus/overwrite header dari inbound request dan menyuntikkan secret server-side. Service wajib memverifikasi JWT walaupun secret benar.

# 04 / Stack dan pengelolaan service

Pakai TypeScript pada frontend dan backend untuk kontrak yang lebih jelas. Gunakan versi paket yang kompatibel dan stabil berdasarkan dokumentasi resmi saat implementasi, bukan menyalin versi lama dari tutorial. Jangan migrasikan tool/arsitektur tanpa alasan.

| Lapisan | Pilihan utama | Batasan implementasi |
|---|---|---|
| Frontend | React + Vite + TypeScript; React Router; TanStack Query; React Hook Form + Zod. | Aplikasi browser; tidak berisi secret. Semua data domain dari Gateway. |
| UI system | Tailwind CSS + shadcn/ui primitives; lucide-react; custom design tokens. | Gunakan primitives yang diadaptasi, bukan dashboard starter ditempel tanpa design. |
| Gateway | Node.js + TypeScript + Express; http-proxy-middleware atau proxy tipis yang diuji. | Port 3000; auth/policy/audit/proxy; limit body, timeout, request ID. |
| Service Nilai | Express + TypeScript; modules/routes/services/repositories. | Port 3001; proses terpisah; hanya nilai, kelas, finalisasi. |
| Service Transkrip | Express + TypeScript; modules/routes/services/repositories. | Port 3002; proses terpisah; data transkrip dan idempotent sync. |
| Database | PostgreSQL lokal + migrations + seed. | Tidak menggunakan SQLite/Neon/Supabase untuk task ini. Hindari schema sync otomatis tanpa migration. |
| Database access | Pilih satu library/ORM yang stabil (mis. pg + SQL terparametrisasi atau ORM yang cocok). | Jangan mencampur dua ORM. Parameterized query, FK, unique constraints, transaksi. |
| Validasi & observability | Zod/Joi untuk input; Pino atau logger terstruktur; Helmet; CORS allowlist; request ID. | Sanitize output, jangan bocorkan stack trace/secret ke UI. |
| Testing | Vitest + Supertest untuk unit/integration; Playwright untuk E2E browser. | Test harus menjalankan app/API nyata lokal, bukan cuma mock semua service. |

## Pengelolaan proses

- Local development harus mudah: satu perintah menjalankan Postgres (Docker Compose jika mempermudah), lalu perintah terpisah untuk gateway, service nilai, service transkrip, dan frontend.
- Sediakan `health`/`ready` endpoints yang tidak membocorkan informasi sensitif. Readiness memverifikasi dependency yang benar-benar dibutuhkan.
- Tangani SIGTERM/SIGINT, tutup HTTP server dan pool database secara graceful; jangan meninggalkan proses atau port yatim.
- Gunakan `.env.example` yang aman; `.env`, private keys, log runtime, test artifacts, dan kredensial lokal harus masuk `.gitignore`.
- Jangan wajibkan Docker untuk semua service jika runtime lokal lebih praktis. PostgreSQL lokal tetap menjadi satu-satunya sumber data. Dokumentasikan dua opsi hanya bila keduanya benar-benar dites.

# 05 / API contract dan siklus token

Pertahankan endpoint wajib dari dokumen. Endpoint tambahan boleh dibuat untuk pengalaman dashboard (misalnya GET /me, GET /dashboard/summary, GET /kelas) tetapi wajib melewati Gateway dan tetap mengikuti policy. Dokumentasikan kontrak request/response dan error code; jangan mengubah arti endpoint rubrik.

| Method + path | Aktor | Aturan inti |
|---|---|---|
| POST /auth/login | Semua user | Kredensial valid + X-Device-ID terdaftar dan dimiliki user. Terbitkan access JWT 10 menit. |
| POST /auth/refresh | Session user | Validasi refresh token tidak expired/revoked; rotasi token; kembali menerbitkan access JWT. |
| POST /auth/logout | Session user | Cabut refresh token/family yang terkait; clear cookie; catat logout. |
| POST /auth/service | Service Nilai | Client ID + secret valid (constant-time comparison bila memungkinkan); terbitkan service JWT 5 menit. |
| GET /nilai | Mahasiswa / dosen | Mahasiswa hanya nilai sendiri; dosen hanya kelas yang diampu. |
| PUT /nilai/:id | Dosen | Kelas diampu, nilai belum final, window akses aktif; validasi rentang nilai. |
| POST /kelas/:id/finalisasi | Dosen | Kelas diampu, jam akses valid, cek konsistensi nilai; idempotent. |
| GET /transkrip/:nim | Mahasiswa / admin | Mahasiswa hanya NIM sendiri; admin dapat melihat semua NIM. |
| POST /transkrip/sinkron | Service Nilai saja | Hanya token service valid; token user biasa harus ditolak. |

## Desain token yang direkomendasikan

- Access JWT: backend membuat token saat login dan refresh; TTL `10m`. Minimal `sub` + `role`; tambahkan `iss`, `aud`, `iat`, `exp`, `jti`, `token_use=access`. Validasi algoritma yang diizinkan secara eksplisit.
- Refresh token: token opaque dengan nilai acak kriptografis, masa berlaku 1 hari, simpan hash-nya di PostgreSQL (bukan plaintext), revocable dan rotate ketika dipakai. Cookie browser sebaiknya HttpOnly + SameSite, dengan proteksi Origin/CSRF yang sesuai.
- Service JWT: TTL `5m`, `role=service`, `token_use=service`, `sub=grades-service`, audience terbatas ke service transkrip atau tujuan yang disetujui. Jangan menggunakan token user untuk komunikasi service.
- Signing key: prefer asymmetric signing (mis. RS256): Gateway menyimpan private key, service hanya memerlukan public key. Buat satu kali melalui script setup dan jangan overwrite key yang sudah ada tanpa perintah eksplisit.
- Storage browser: jangan menaruh access/refresh token di localStorage. Simpan access token di memory dan refresh token di HttpOnly cookie; jelaskan batasan desain dan CORS credentials.

## Contoh konfigurasi (nilai rahasia sengaja tidak diisi)

```
ACCESS_TOKEN_TTL=10m
REFRESH_TOKEN_TTL=1d
SERVICE_TOKEN_TTL=5m
ACCESS_WINDOW_START=07:00
ACCESS_WINDOW_END=21:00
ACCESS_TIMEZONE=Asia/Jakarta
DATABASE_URL=postgresql://...@localhost:5432/zero_trust_db
```

# 06 / PostgreSQL dan model data

Gunakan PostgreSQL lokal, migrations yang dapat diulang dengan aman, constraints, foreign keys, serta seed yang deterministik. Skema di bawah adalah rancangan awal; agent harus menyesuaikan dengan kebutuhan yang ditemukan saat membuat ERD dan tetap mendokumentasikannya.

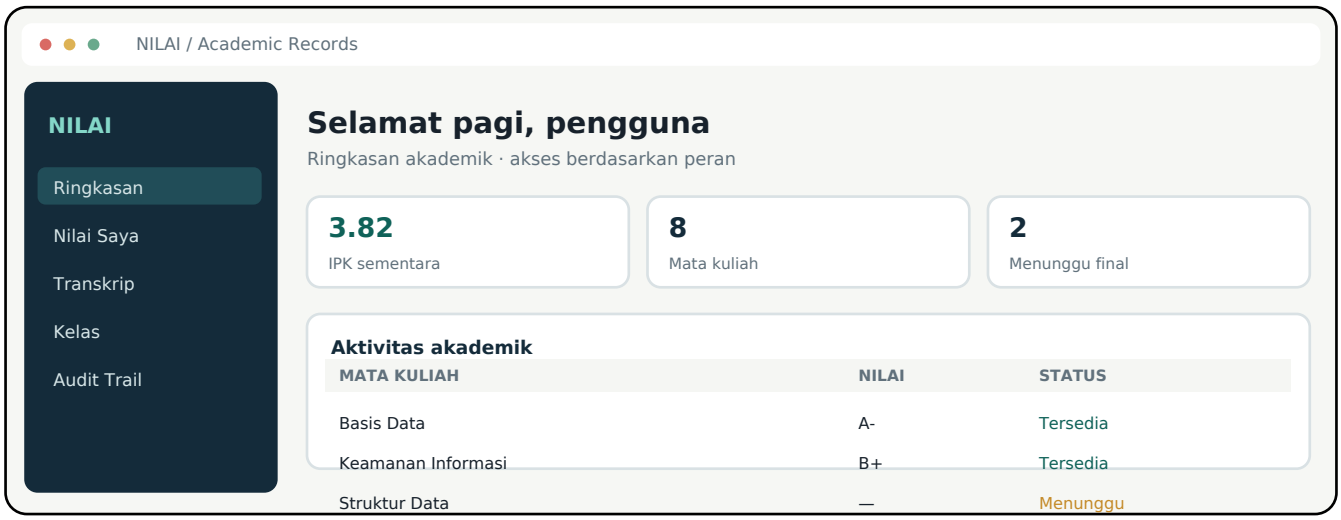
| Tabel | Kolom/relasi penting | Catatan integritas |
|---|---|---|
| users | id UUID, username/email, full_name, role, password_hash, is_active, timestamps | Unique username/email; role dibatasi enum/check. Password hash Argon2id atau bcrypt cost wajar; bukan plaintext. |
| devices | id, user_id FK, device_id unique, label, is_active, last_seen_at | Setiap seed user punya satu device; cek ownership device per user. |
| courses | id, code unique, name, credits | Validasi kredit dan code. |
| classes | id, course_id FK, lecturer_id FK users, term, section, is_finalized, finalized_by, finalized_at | Index lecturer_id; finalisasi dilindungi transaksi dan ownership. |
| enrollments | id, class_id FK, student_id FK users | Unique (class_id, student_id); hanya role mahasiswa yang bisa terdaftar. |
| grades | id, class_id FK, student_id FK, score numeric, updated_by FK, timestamps | Unique (class_id, student_id); CHECK skor 0..100; update diblok saat kelas final. |
| transcript_entries | id, student_id FK, class_id FK, course_id FK, score, grade_letter, synced_at, source_version | Unique per mahasiswa + kelas; sinkronisasi upsert idempotent. |
| refresh_sessions | id, user_id FK, token_hash, expires_at, revoked_at, replaced_by_id, created_at | Unique token_hash; jangan simpan raw refresh token. |
| service_clients | id, client_id unique, secret_hash/secure verifier, allowed_scope, is_active | Secret disimpan aman; endpoint service membatasi client yang valid. |
| audit_events | id, occurred_at, subject_id, subject_type, method, path, device_id, decision, reason_code, request_id | Index waktu/subject/decision; jangan simpan token, password, gateway secret. |
| outbox_events (nilai plus) | id, event_type, aggregate_id, payload terbatas, status, attempts, created_at, processed_at | Opsional: retry sinkronisasi jika Service Transkrip down; cegah kehilangan event setelah finalisasi. |

## Migration dan transaction rules

- Pisahkan schema/table ownership secara logis. Untuk nilai plus keamanan, gunakan role database terpisah bagi gateway, service nilai, service transkrip, dan migration/seed; berikan hak minimum yang benar-benar dibutuhkan.
- Gunakan transaksi untuk update nilai dan finalisasi. Kunci kelas/row terkait jika diperlukan untuk mencegah dua request finalisasi/update berlomba.
- Gunakan SQL parameterized atau query builder yang aman. Jangan membangun query melalui konkatenasi string dari input user.
- Sinkronisasi transkrip harus idempotent: request ulang dengan event/kelas yang sama tidak membuat entry duplikat. Catat status sukses/gagal dan siapkan retry manual/script yang aman.
- Seed minimal: 1 admin, 2 dosen, 3 mahasiswa, 2 kelas berbeda yang diampu dua dosen berbeda, enrollment + grades + device tiap user. Semua password demo tetap di-hash.

# 07 / Arahan visual: premium, bukan AI slop

Bangun aplikasi bernama sementara NILAI — Academic Records (boleh mengganti nama jika punya alasan kuat). UI adalah alat kerja akademik yang tenang, presisi, kredibel, dan berorientasi data. Bukan landing page marketing, bukan template dashboard mentah, dan bukan mockup statis.




## Arah art direction

| Aspek | Arahan visual |
|---|---|
| Kepribadian | Editorial + product UI profesional: tenang, terukur, tepercaya, ringan. Gunakan ruang kosong untuk hierarki, bukan efek dekoratif. |
| Palet awal | Canvas warm-gray / off-white; teks ink/navy; accent teal/emerald yang terkendali; status sukses, peringatan, gagal dibedakan juga dengan label/icon. Kontras WCAG AA. |
| Tipografi | Pilih satu keluarga sans yang sangat terbaca (mis. Geist/Inter/system fallback). Gunakan 3–4 skala yang konsisten; judul ringkas; label dan tabel jangan terlalu kecil. |
| Layout | Sidebar yang rapi, topbar minimal, max-width konten konsisten, tabel nilai sebagai fokus utama, filter jelas, breadcrumb seperlunya. Responsive dari 360 px hingga desktop. |
| Komponen | Data table yang benar (sorting/filtering/pagination bila relevan), modal konfirmasi finalisasi, toast, badge status, skeleton, empty/error state, form inline yang accessible. |
| Motion | Transisi halus 120–200 ms bila membantu pemahaman. Hormati prefers-reduced-motion. Tanpa parallax/glassmorphism/animasi demi sensasi. |
| Bahasa | Gunakan Bahasa Indonesia yang natural, istilah akademik konsisten, pesan error menjelaskan tindakan berikutnya tanpa membocorkan detail internal. |

> Larangan anti-template: jangan gunakan gradient ungu generik di seluruh layar, deretan 4 kartu statistik yang tidak memberi keputusan, emoji sebagai icon utama, blob dekoratif, chart palsu, lorem ipsum, angka yang tidak berasal dari API, ataupun tombol yang tidak melakukan apa-apa.

## Referensi yang wajib dibuka sebelum styling

- shadcn/ui Dashboard Blocks — pelajari hierarki sidebar, tabel dan komposisi, lalu buat identitas visual sendiri.
- Rad UI — referensi primitive yang accessible/headless; jangan mengorbankan aksesibilitas demi visual.
- Pulse dashboard concept dan TaskFlow dashboard concept — referensi visual/inspirasi struktur saja; jangan menyalin persis atau menyalin aset tanpa izin.

## Proses desain

1. Buka reference links dan skill UI yang terpilih. Ambil screenshot atau catat pola yang relevan dan mengapa.
2. Tulis mini design brief: persona, skenario utama, informasi terpenting, status/error state, dan tokens warna/spasi/typography.
3. Buat satu layar utama per role sebelum mengembangkan seluruh route. Review visual di browser pada desktop dan mobile.

# 08 / Frontend functional & quality bar

Frontend harus terhubung ke API Gateway nyata. Data utama tidak boleh hard-coded; mock hanya boleh dipakai dalam storybook/preview terpisah dan harus jelas bukan production data. Role UI adalah convenience, bukan security boundary: backend memutuskan hak akses.

| Role / area | Minimum experience yang harus berfungsi |
|---|---|
| Login | Username/email + password; pilihan akun demo hanya pada local/demo mode; device ID terdaftar dipasang ke X-Device-ID; loading, invalid credentials, device rejected, lockout/rate limit feedback. |
| Mahasiswa | Dashboard ringkas; daftar nilai sendiri; detail mata kuliah/semester; transkrip diri sendiri; empty/loading/error states. Tidak ada input NIM orang lain di UI mahasiswa. |
| Dosen | Daftar kelas yang diampu; gradebook untuk peserta kelas; form edit skor valid; indikator kelas final; modal konfirmasi finalisasi dengan dampak jelas; locked state setelah final. |
| Admin | Pencarian/filter transkrip semua mahasiswa; loading dan hasil kosong; akses route disesuaikan policy. Optional: audit trail read-only untuk demonstrasi. |
| Global | Header/sidebar konsisten; user role/name; logout; session-expired handling; toast/action feedback; pagination/empty/error state; 404 + denied page; keyboard navigation. |

## Route/page plan

- `/login` — form login dan local demo shortcuts. Jangan pernah menampilkan password/secret pada build non-demo.
- `/app` — route shell yang memilih dashboard berdasarkan role setelah `/me`/session established.
- `/app/nilai` — tampilan nilai; dosen dapat beralih ke kelas yang diampu, mahasiswa hanya melihat miliknya.
- `/app/transkrip` — tampilan transkrip untuk mahasiswa atau pencarian NIM admin; API tetap menerapkan authorization.
- `/app/kelas/:id` — gradebook/finalisasi untuk dosen yang memang pemilik kelas.
- `/app/audit` — nilai plus opsional: audit-event viewer admin-only, read-only, dengan alasan keputusan; jangan prioritaskan sebelum rubrik inti lulus.

## Interaksi UI yang harus nyata

- Form menggunakan schema validation dan menampilkan error per field. Tombol disabled saat request berjalan; cegah double submit.
- API error mapping: 400 validation, 401 unauthenticated/expired, 403 denied, 404 missing resource, 409 conflict/finalized, 429 rate limit, 5xx unavailable. Tampilkan pesan aman dan jelas.
- Saat access JWT expired, lakukan refresh sekali secara terkendali; jika gagal, bersihkan memori session dan kembalikan user ke login. Hindari infinite refresh loops.
- A11y: landmark semantik, label form, focus visible, modal trap+restore focus, keyboard usable, color contrast AA, table accessible, status tidak bergantung pada warna saja.
- Sediakan responsive behavior, reduced motion, dan test pada 360x800, 768px, dan 1440px; jangan menganggap responsive selesai karena hanya memakai Tailwind breakpoints.

# 09 / Skill pack terpilih dari instalasi user

Inventory yang diberikan berupa nama folder di `C:\Users\FATHIR\.agents\skills`. Nama-nama di bawah ditemukan pada daftar itu. Agent tetap harus memverifikasi folder dan membaca file instruksi aktual sebelum menganggap skill tersedia/berhasil dipakai.

| Tahap | Skill/folder yang diprioritaskan | Gunakan untuk |
|---|---|---|
| Orchestrasi | `antigravity-skill-orchestrator`, `before-you-build` | Memetakan skill sesuai fase; melakukan discovery sebelum perubahan besar. |
| Visual/UX | `high-end-visual-design`, `frontend-design`, `ui-ux-pro-max`, `web-design-guidelines`, `frontend-ui-engineering` | Art direction, anti-template, hierarchy, responsive patterns, review kualitas UI. Pilih 2–3 utama, jangan semua dipanggil tanpa tujuan. |
| Frontend architecture | `frontend-architecture`, `frontend-dev-guidelines`, `frontend-api-integration-patterns`, `accessibility-compliance-accessibility-audit`, `frontend-lighthouse` | Komponen/routes/data fetching/API contract, accessibility, performance. |
| System/backend | `backend-architect`, `backend-dev-guidelines`, `nodejs-backend-patterns`, `software-architecture`, `microservices-patterns` | Boundaries antar-process, module/service design, robust Node patterns. |
| Security/API/auth | `backend-security-coder`, `api-security-best-practices`, `api-security-testing`, `auth-implementation-patterns`, `security-requirement-extraction` | Token/authz, policy enforcement, secret handling, threat-model-to-controls, secure API tests. |
| PostgreSQL | `postgres-best-practices`, `postgresql`, `postgresql-optimization` bila query lambat | Schema/migrations/indexes/transactions/constraints dan query plan jika perlu. |
| Service runtime | `docker-expert`, `container-security-hardening` bila Compose/container dipakai | Local orchestration, health checks, network/secret separation, graceful shutdown. |
| Test & E2E | `playwright-skill`, `webapp-testing`, `e2e-testing-patterns`, `testing-qa`, `unit-testing-test-generate`, `javascript-testing-patterns` | Test pyramid, browser flows, edge cases, deterministic fixtures, regression coverage. |
| Security verification | `security-audit`, `security-auditor`, `security-scanning-security-sast`, `web-security-testing`, `idor-testing`, `sql-injection-testing` | Audit after implementation, authz/IDOR/SQL injection checks, dependency/SAST findings. |
| Final review | `code-review-excellence`, `code-review-and-quality`, `acceptance-orchestrator` | Independent review, requirements traceability, final acceptance gate. |

> Gunakan seperlunya, bukan semuanya sekaligus. Minimal gunakan satu skill visual, satu arsitektur/backend, satu security, satu PostgreSQL, satu Playwright/E2E, dan satu review. Skill tambahan digunakan ketika ada alasan.

Skill tersedia bukan berarti otomatis kompatibel dengan proyek. Bila skill menyuruh memakai framework/ORM berbeda dari keputusan proyek, bandingkan dengan requirement dan dokumentasikan keputusan; jangan mengubah stack tanpa justifikasi.

# 10 / Cara pakai skill: instruksi operasional

Jangan hanya menulis “pakai skill X” pada laporan. Baca instructions file-nya, lalu gunakan skill saat fase yang relevan. Jika platform agent mendukung pemanggilan skill langsung, gunakan mekanisme native tersebut. Jika tidak, lakukan inspeksi langsung pada file skill.

## Langkah wajib

1. Jalankan `Get-Location`, `git status --short`, dan inventaris file project. Jangan mulai generate ulang seluruh proyek tanpa mengetahui keadaan awal.
2. Verifikasi skill di folder `C:\Users\FATHIR\.agents\skills\`. Biasanya ada `SKILL.md`; jangan berasumsi nama file sebelum diperiksa.
3. Baca seluruh bagian yang relevan dari `SKILL.md`, termasuk assets/references/checklists yang dirujuk. Jika skill mengandung perintah/tools, gunakan yang sesuai dan aman.
4. Sebelum tiap fase, nyatakan skill yang dipakai + tujuan spesifik. Setelah fase, catat file/keputusan/perbaikan apa yang dipengaruhi skill.
5. Jika folder atau SKILL.md tidak ada, jangan pura-pura memakai skill. Cari kandidat dari inventory, pilih pengganti yang tersedia, dan catat substitusinya.

## PowerShell untuk memverifikasi dan membuka instruksi

```
Set-Location $HOME\.agents\skills\frontend-design
Get-ChildItem -Force
Get-ChildItem -Recurse -Filter SKILL.md | Select-Object FullName
Get-Content .\SKILL.md -Raw

# Ulangi dengan nama folder skill lain; jangan copy secret ke terminal output.
```

## PowerShell untuk mencari skill kandidat

```
Get-ChildItem "$HOME\.agents\skills" -Directory |
  Where-Object Name -Match "design|frontend|backend|security|postgres|playwright|e2e|testing|audit" |
  Select-Object -ExpandProperty Name
```

## Prompt mikro yang harus diikuti per fase

> Design: “Baca skill visual terpilih dan web-design-guidelines. Buka referensi eksternal, tentukan desain sistem serta kriteria anti-AI-slop sebelum implementasi. Tunjukkan wireframe/token keputusan dan terapkan pada kode.”

> Backend/security: “Baca backend-architect, nodejs-backend-patterns, backend-security-coder dan api-security-best-practices. Petakan setiap ancaman ke kontrol kode + test negatif. Jangan anggap role-check di frontend sebagai authorization.”

> QA: “Baca playwright-skill/webapp-testing/e2e-testing-patterns dan security-audit. Jalankan unit, integration, security abuse cases, lalu Playwright E2E. Ambil screenshot dan perbaiki temuan sebelum laporan akhir.”

# 11 / Security baseline dan threat model

Terapkan prinsip tugas secara eksplisit: never trust, always verify; least privilege; assume breach. Boleh menambah kontrol, tetapi jangan membuat implementasi yang lebih rumit dan tidak teruji. Semua aturan keamanan harus memiliki lokasi kode dan bukti pengujian.

| Threat / bypass attempt | Kontrol minimal | Automated check |
|---|---|---|
| Token hilang / palsu / expired | Fail closed; verify signature + issuer + audience + exp + token_use. | 401/403, service handler tidak dijalankan; audit event deny. |
| User membaca data orang lain (IDOR) | Ownership filter di backend/Policy Engine dan query repository, bukan hanya di UI. | Mahasiswa A minta transcript/grades B -> deny; test juga path/parameter alternatif. |
| Role escalation | Role berasal dari JWT valid dan sumber user yang tepercaya; tidak menerima role dari body/query. | Mahasiswa kirim role=admin di body -> tidak berpengaruh. |
| Spoof X-Device-ID | Periksa device aktif milik subject hasil token; login memeriksa device milik akun yang cocok. | Device milik user lain/unknown/disabled -> deny. |
| Direct service access | Service independently verifies gateway secret + JWT; internal service not documented as public API. | No secret -> deny; wrong secret -> deny; valid secret + bad JWT -> deny. |
| Client injects gateway secret | Gateway strips any inbound X-Gateway-Secret and overwrites from server env. | Send malicious/fake header; verify secret reaching service is gateway-owned. |
| User invokes service sync | Endpoint requires verified service token, correct audience/scope/client; not merely role string. | Valid user JWT -> deny. Expired service token -> deny. |
| SQL injection / invalid body | Parameterized queries, allowlisted sort/filter fields, schema validation, bounded payload. | SQLi payload does not change query semantics; invalid payload gets 400. |
| Refresh replay / logout bypass | Hash refresh token in DB, rotate on use, revoke old token, TTL 1d, logout revokes. | Old token reuse/after logout/after expiry fails. |
| Audit leaks credentials | Allow-listed structured fields; redact Authorization/Cookie/secret/body as appropriate. | Inspect stored logs for canary secrets/tokens; none present. |
| Time-window abuse | Server-side time check using configured Asia/Jakarta timezone; never trust client clock. | Inject test clock at 06:59, 07:00, 20:59, 21:00, 21:01 per defined boundary. |

## Tambahan penting

- Rate-limit login dan token endpoints; gunakan generic error message agar user enumeration berkurang, namun tetap tampilkan penolakan device dengan pesan aman.
- Hash password menggunakan Argon2id atau bcrypt yang dikonfigurasi aman; constant-time compare untuk client secret jika implementasi memungkinkan.
- Jangan log raw Authorization/Cookie; batasi CORS ke origin frontend lokal tertentu; aktifkan Helmet dan content security policy yang cocok dengan Vite app.
- Jangan menyimpan credentials di frontend bundle, `VITE_*`, commit, screenshot, atau README publik. `VITE_*` adalah variabel yang bisa masuk bundle browser.
- Periksa dependency audit dan static analysis. Temuan tidak boleh disembunyikan; perbaiki high/critical atau dokumentasikan alasan dan mitigasi.

## Rujukan kontrol security

NIST SP 800-207 — Zero Trust Architecture · OWASP ASVS · OWASP JWT Cheat Sheet

# 12 / Testing strategy — wajib komprehensif

Testing tidak boleh disederhanakan menjadi “npm run build sukses”. Buat test pyramid dan bukti hasil. Gunakan clock injection atau frozen time pada test untuk memverifikasi access window secara deterministik; jangan berharap test waktu berjalan lulus kebetulan.

| Lapisan | Tools / pendekatan | Cakupan |
|---|---|---|
| Static / quality | TypeScript strict, ESLint, formatter, dependency audit, secret scan/SAST jika tersedia. | Type errors, lint, unused code, risky deps, committed secrets, unsafe patterns. |
| Unit | Vitest (atau test runner konsisten yang cocok dengan repo). | Policy allow/deny, ownership, time window, role, token claims, input validation, grade calculation/status. |
| Integration API | Supertest terhadap Gateway dan service nyata; test database PostgreSQL khusus. | Login/refresh/logout; gateway proxy; headers; database constraints; transaction dan sync. |
| Security negative tests | API security test cases + IDOR/SQLi/broken access control. | Auth bypass, role escalation, direct port access, secret spoof, token confusion, refresh reuse. |
| Contract / service tests | Validasi request/response schema dan error status; test downstream failure. | Service Nilai -> Gateway -> Service Transkrip; timeout/retry/idempotency/failure states. |
| Browser E2E | Playwright Chromium; run frontend + gateway + services + Postgres. | Login sebagai role berbeda, navigation, nilai mahasiswa, grade edit/finalization, transkrip, logout/expired session. |
| Visual/responsive/A11y | Playwright screenshots, axe/accessibility skill bila tersedia, keyboard/manual browser review. | 360px/mobile, tablet, desktop; focus, contrast, modal, loading/error/empty states, no clipping. |
| Smoke / operations | Health/readiness checks + startup/shutdown exercise. | Ports, env validation, DB unavailable, service unavailable, startup order, graceful shutdown. |

## E2E journeys yang wajib berjalan

1. Mahasiswa demo login → lihat nilai/transkrip sendiri → ubah URL/parameter ke NIM lain → API menolak → logout → refresh session tak bisa dipakai lagi.
2. Dosen 1 login → buka kelas miliknya → ubah nilai valid → nilai tersimpan → finalisasi → gradebook menjadi read-only → transkrip tersinkron melalui Service Transkrip.
3. Dosen 1 mencoba kelas Dosen 2 → ditolak. Dosen mencoba update saat window ditutup atau nilai final → ditolak.
4. Admin login → baca transkrip mahasiswa berbeda → berhasil. Admin tidak memperoleh hak edit nilai jika tidak tercantum di matriks policy.
5. API attacker tests: tanpa JWT, JWT palsu/expired, device unknown/disabled, token user ke endpoint service sync, gateway secret kosong/salah, direct service request.
6. Test sinkronisasi: Service Transkrip sementara down; finalisasi tidak kehilangan event. Retry idempotent memulihkan sinkron tanpa duplikasi.

> Laporan testing wajib berisi command yang dijalankan, jumlah pass/fail/skip, test failure yang diperbaiki, screenshot UI, serta temuan yang tersisa. Jangan mengklaim 100% tanpa report nyata.

# 13 / Urutan implementasi dan quality gate

| Gate | Kerja yang harus selesai | Tidak boleh lanjut sebelum... |
|---|---|---|
| G0 — Discovery | Audit workspace, runtime, skill inventory, task PDF. Tulis `docs/implementation-plan.md`, ADR ringkas, risk list. | Tidak ada file pengguna tertimpa; seluruh requirement memiliki acceptance test. |
| G1 — Design/API contract | ERD, service boundaries, API contracts, role-policy matrix, UI tokens/wireframe, threat model. | Gateway/Service Nilai/Service Transkrip benar-benar terpisah; PostgreSQL dipilih. |
| G2 — DB/Auth baseline | Migration + deterministic seed, password hashing, key generation, login/access JWT/refresh/logout/device. | Auth unit + integration tests lulus; keys tidak ter-commit. |
| G3 — Gateway + service security | PEP proxy, policy engine, audit logging, secret injection, independent service verification. | Direct-access dan invalid-token negative tests lulus. |
| G4 — Business workflow | Nilai GET/PUT/finalisasi; transcript GET/sync; service token; transaction + idempotency. | Role/ownership/time/finalized-state tests lulus; sync punya failure strategy. |
| G5 — Premium frontend | Login, role shell, dashboards, gradebook, transcript, responsive states, API integration. | No fake functional buttons; UI navigates and consumes real endpoints. |
| G6 — Full test & hardening | Unit/integration/security/E2E/a11y/responsive/performance, review dependency, fix findings. | All blocking tests pass; no high-risk known issue without a documented reason. |
| G7 — Handover | README, `.env.example`, migrations/seed, endpoint docs, test report, screenshots, demo script. | A reviewer can clone/setup/run/test using only written instructions. |

## Mandatory project documentation

- `README.md`: prerequisites, exact setup commands for Windows PowerShell, env setup, PostgreSQL database/user creation, migrations/seed, run commands per app, test commands, seed accounts/device IDs, common errors, ports.
- `docs/architecture.md`: architecture diagram, responsibilities, request sequences, network/process layout.
- `docs/security-model.md`: trust boundaries, policy matrix, threats + controls, token lifecycle, known limitations (especially device ID is not cryptographic attestation).
- `docs/api.md` or OpenAPI: endpoints, payload examples, auth requirements, status codes, sample success/deny flows.
- `docs/testing-report.md`: test commands and actual results, test scenario matrix, E2E screenshots, remaining issues.
- `docs/skills-used.md`: each skill name, location verified, when/how used, concrete files/decisions influenced, substitute if unavailable.
- `docs/demo-script.md`: plan 6–9 minute video covering required mechanisms and the source code location for each.

# 14 / Nilai plus — setelah rubrik lulus

Pilih tambahan yang meningkatkan kualitas engineering tanpa mengganggu jalur wajib. Implementasikan 1–3 yang bernilai tinggi dan benar-benar testable, bukan banyak fitur setengah jadi.

| Nilai plus | Manfaat | Prioritas |
|---|---|---|
| Audit Trail UI admin-only | Menampilkan allow/deny, waktu, subject, endpoint, device, reason_code; filter/search; tidak ada secret/token. | Tinggi setelah keamanan inti. |
| Outbox + retry sync | Finalisasi merekam event transkrip di transaction; background/command worker retry ketika downstream down; idempotency key mencegah duplikasi. | Tinggi jika dapat dites. |
| DB least-privilege roles | Migration role terpisah untuk service dengan grants minimum, bukan semua service memakai superuser. | Tinggi untuk keamanan. |
| Policy decision trace | Audit internal menyatakan policy rule/reason code yang memicu deny, tanpa data berlebihan. | Sedang; berguna saat demo. |
| OpenAPI + Postman collection | Kontrak endpoint dan request demo siap diimpor. | Tinggi untuk maintainability/demonstrasi. |
| Observability lokal | Structured logs, request ID end-to-end, readiness endpoints; ringkasan service health admin/dev only. | Sedang; jangan tampilkan rahasia/stack trace. |
| CI workflow | Install, lint, typecheck, unit/integration tests, build frontend. E2E bisa job terpisah bila perlu. | Tinggi bila tidak menyulitkan local run. |
| A11y and visual regression | Playwright screenshot baseline + keyboard/contrast review. | Tinggi untuk kualitas frontend. |

## Ide tambahan yang harus dihindari

- Jangan menambah layanan Redis, message broker, Kubernetes, cloud deployment, LLM, atau microservices lain tanpa kebutuhan. Task harus berjalan lokal dan mudah dinilai.
- Jangan membangun dashboard metrik dengan angka random. Semua indikator harus diturunkan dari data nyata atau diberi label demonstrasi yang eksplisit.
- Jangan mengekspos service ports sebagai API publik, menanam secret ke frontend, menyimpan password plaintext, atau menganggap URL tersembunyi sebagai kontrol akses.
- Jangan mengoptimalkan premature. Prioritaskan correctness, security, readability, tests, lalu performa yang diukur.

> Jika ada trade-off, prioritas berurutan: (1) memenuhi rubrik, (2) keamanan, (3) konsistensi data, (4) testability, (5) UX visual, (6) fitur plus. Jangan menukar 1–4 demi animasi atau efek visual.

# 15 / Prompt eksekusi final untuk agent

Instruksi berikut adalah mandat langsung, bukan sekadar saran. Jalankan proyek sampai selesai dan verifikasi hasilnya.

> Mulai sekarang: audit workspace dan PDF tugas; inventaris skill yang tersedia; baca skill yang relevan; susun satu rencana singkat dan requirement-to-test matrix; implementasikan sistem; jalankan PostgreSQL lokal; jalankan seluruh service dan frontend; selesaikan test unit/integration/security/E2E; review screenshot UI; perbaiki bug; baru susun laporan akhir.

## Sebelum mengakhiri sesi, kamu wajib menjawab dengan format ini:

1. Ringkasan implementasi: fitur selesai dan lokasi penting di repo.
2. Arsitektur aktual: ports/processes/database, flow Gateway → Policy → Service, serta alasan keputusan teknis.
3. Setup & run: perintah persis untuk Windows PowerShell, PostgreSQL setup, env, migration, seed, lalu start setiap proses.
4. Demo credentials: username, password demo, X-Device-ID, role; jelaskan bahwa kredensial hanya untuk lokal dan jangan dipakai di internet.
5. Skill audit trail: skill yang benar-benar dibaca/dipakai, path SKILL.md yang diverifikasi, perubahan yang dipengaruhi, dan skill yang tidak tersedia.
6. Testing evidence: command, pass/fail/skip, Playwright screenshot path, test security yang dijalankan, dan temuan yang belum selesai.
7. Acceptance matrix: setiap requirement wajib -> file implementasi -> automated test/demo evidence.
8. Residual risks: apa yang belum diuji, batasan local demo, dan rekomendasi berikutnya; tidak boleh menyembunyikan test gagal.

## Final acceptance checklist

- Frontend premium, responsive, data-driven, tanpa elemen dekoratif generik atau tombol palsu.
- Frontend hanya mengakses Gateway; service punya proses terpisah dan memverifikasi secret + JWT.
- PostgreSQL lokal, migration repeatable, seed requirement lengkap, no plaintext secrets/passwords.
- JWT 10m + refresh revocable 1d + service token 5m; key dibuat sekali dan disimpan lokal/ignored.
- Semua policy role, ownership, device, time-window, finalisasi, service-to-service, audit logging diproteksi dan dites.
- Unit + integration + security negative + Playwright E2E + visual/responsive review telah dijalankan; kegagalan nyata dilaporkan.
- README dan demo script memungkinkan dosen mengulang demo dari environment lokal.

# 16 / Reference desk

Buka sumber resmi sebelum memutuskan detail framework/security. Design shots adalah referensi visual, bukan lisensi untuk menyalin UI/aset. Gunakan dokumen tugas sebagai kontrak requirement utama.

## Design & frontend

shadcn/ui dashboard blocks — ui.shadcn.com/blocks?category=dashboard
Pola dashboard/sidebar/tabel yang bisa diadaptasi; jangan mempertahankan visual starter secara mentah.

Rad UI — rad-ui.com/docs/first-steps/introduction
Referensi komponen headless/accessibility dan disiplin design-system.

Pulse SaaS Analytics dashboard (Dribbble) — open reference
TaskFlow Analytics dashboard (Dribbble) — open reference
Inspirasi komposisi/spacing saja; jangan clone.

## Security & architecture

NIST SP 800-207: Zero Trust Architecture — NIST publication
Landasan prinsip Zero Trust dan proteksi resource, bukan trust berdasar lokasi jaringan.

OWASP ASVS — OWASP project
Katalog requirement keamanan aplikasi untuk merumuskan checklist verifikasi.

OWASP JWT Cheat Sheet — read guidance
Validasi JWT, klaim, revocation considerations, risiko logging dan storage.

## Testing

Playwright Test documentation — playwright.dev/docs/intro
Framework end-to-end untuk browser modern; gunakan test report, trace, dan screenshot ketika relevan.

## Keterangan sumber

Persyaratan role, endpoint, port, masa berlaku token, verifikasi device, jam akses, audit log, seed, dan luaran berasal dari PDF tugas praktikum Zero Trust yang diberikan pengguna. Stack, schema draft, token-hardening, UI direction, skill workflow, dan test matrix dalam brief ini merupakan arahan implementasi yang disarankan untuk memenuhi tugas serta menambahkan nilai engineering.

**END OF BRIEF**

**Bangun dengan disiplin: desain punya alasan, setiap kontrol punya test, dan setiap klaim punya bukti.**